

CUDA Application Programming Interface

Parallel and Heterogeneous Computing

David Mayerich

Scalable Tissue Imaging and Modeling (STIM) Laboratory

Department of Electrical and Computer Engineering

Cullen College of Engineering

University of Houston

UNIVERSITY of **HOUSTON** | ECE

CUDA API and Language

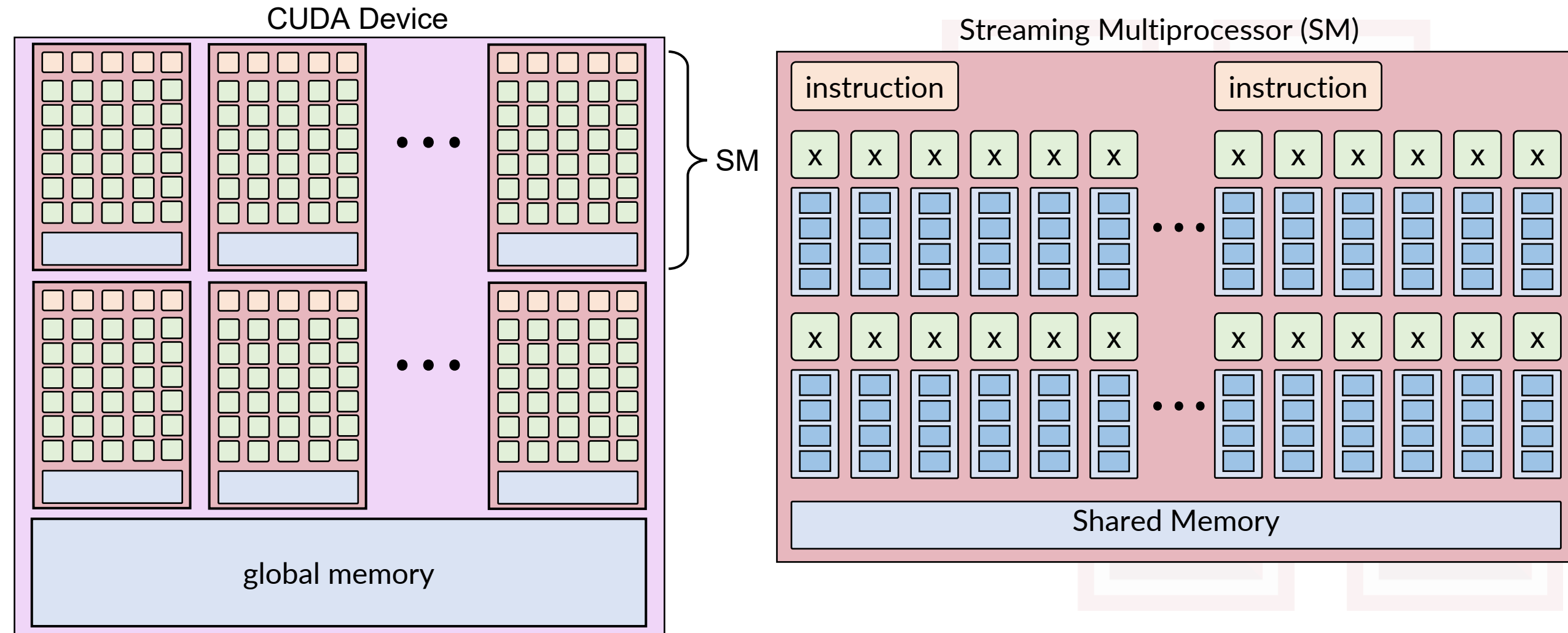
Application Programming Interface (API)

- C/C++ functions that query and initialize GPU features
- Allocating space in device memory, copying data, getting device information
- Setting up special types of memory on the GPU

CUDA language

- Implement code executing on CUDA devices
- ANSI C is a subset of CUDA
- Additional CUDA keywords:
 - Specify regions of data-parallel code
 - Describe how parallel regions are executed
 - Define how resources are allocated

CUDA Device Architecture

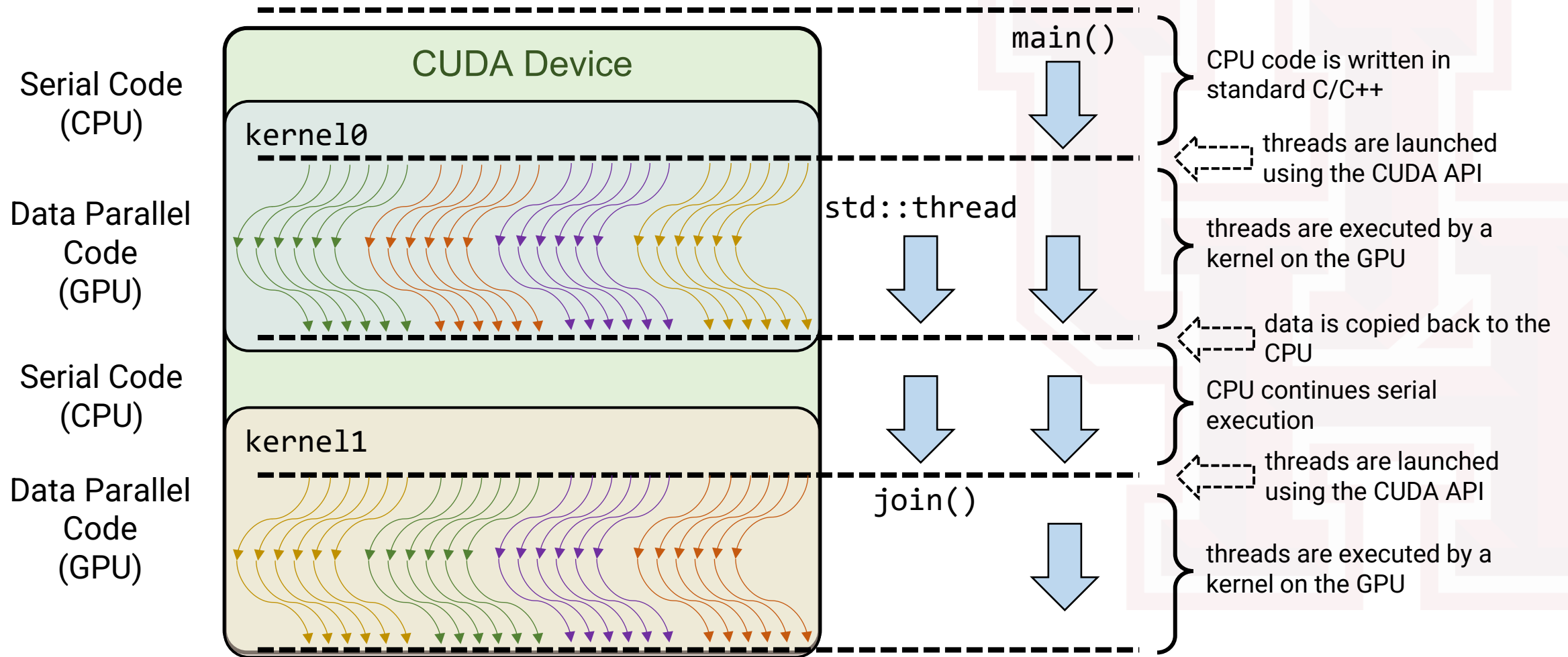


CUDA Kernels

- A kernel is a block of code executed in parallel on a CUDA device
- Similar to a callback function for an `std::thread`

```
__global__ void mmult(float* A, float* B, float* C, int N) {  
    int row = blockIdx.y * blockDim.y + threadIdx.y;  
    int col = blockIdx.x * blockDim.x + threadIdx.x;  
    if(row < N && col < N) {  
        float sum = 0.0f;  
        for(int k = 0; k < N; k++) {  
            sum += A[row * N + k] * B[k * N + col];  
        }  
    }  
}
```

CUDA Threading (and CPU Multithreading)



CUDA API – Device Properties

- get the number of devices (GPUs) available

- code can be run on multiple devices

cudaError_t cudaGetDeviceCount(**int*** n) //get # of CUDA compatible devices

- the programmer can control what devices execute code

cudaError_t cudaGetDevice(**int*** d) //get the current device #

cudaError_t cudaSetDevice(**int** d) //send future commands to this device

- check for compatibility

- test for compute capability – determine what features the device has
- test for available memory – specifies how much data can be copied to the device

cudaError_t cudaGetDeviceProperties(**cudaDevProp*** prop, **int** dev)

Device Properties

- *global memory* – defines how large of a problem you can solve with a single kernel
- *compute capability* – specifies the set of features that the GPU is capable of
 - 1.1 atomic operations were introduced in (2006)
 - 1.3 64-bit floating point operations were introduced in (2008 – 2009)
 - 6.0 64-bit atomic operations (2016)

```
struct cudaDeviceProp{  
    char name[256];  
    size_t totalGlobalMem;  
    ... //includes several important GPU features  
    int major;  
    int minor;  
    ... //more GPU features  
}
```

Error Checking

CUDA API functions return a `cudaError_t` value

- Check This Every Time!

<code>cudaError_t</code>	=	
<code>cudaSuccess</code>		//operation successful
<code>cudaErrorMemoryAllocation</code>		//not enough memory
<code>cudaErrorInvalidValue</code>		//a parameter is wrong
<code>cudaErrorInvalidDevicePointer</code>		//didn't allocate memory?

- Seriously Check It. Trust me.

```
cudaError_t error;  
error = cudaMalloc(gpuPtr, N * sizeof(float));  
if( error != cudaSuccess){  
    throw std::runtime_error("Error allocating memory");  
}
```

Error Checking Macros

```
static void HandleError( cudaError_t err, const char *file, int line ) {  
    if (err != cudaSuccess) {  
        std::cout<<cudaGetErrorString(err)<<" in "<< file <<" at line "<< line;  
    }  
}  
#define HANDLE_ERROR( err ) (HandleError( err, __FILE__, __LINE__ ))
```

- Example:

```
HANDLE_ERROR( cudaMalloc(gpuPtr, N * sizeof(float)) );
```

- will display a string describing the error, along with the file and line number where the error occurred

```

void main(int argc, char* argv[]){
    int N;                                //length of an array
    ...                                  //get array length
    float* x;                             //declare a pointer to main memory
    x = (float*) malloc(N * sizeof(float)); //allocate an array in main memory
    ...                                  //load array values
    float* gpu_x;                         //declare another pointer (will point to device memory)
    HANDLE_ERROR(                         //error checking
        cudaMalloc(&gpu_x, N * sizeof(float)) //allocate memory on device
    );                                    //will return an error or cudaSuccess
    HANDLE_ERROR(
        cudaMemcpy(gpu_x, x, N * sizeof(float), cudaMemcpyHostToDevice)
    );
    //do whatever you want to the array gpu_x on the device

    //copy the array back to main memory
    HANDLE_ERROR(
        cudaMemcpy(x, gpu_x, N * sizeof(float), cudaMemcpyDeviceToHost)
    );
}

```

CUDA API – Basic Framework

- 1) Query for compatible devices

```
cudaError_t cudaGetDeviceCount(int* num)
```

```
cudaError_t cudaGetDeviceProperties(cudaDevProp* prop, int dev)
```

- 2) allocate memory on the CUDA device

```
cudaError_t cudaMalloc(void** devPtr, size_t size)
```

- 3) copy data from main memory to the CUDA device

```
cudaError_t cudaMemcpy(void* dest,  
                        const void* source,  
                        size_t size,  
                        cudaMemcpyKind type)
```

- 4) launch the kernel

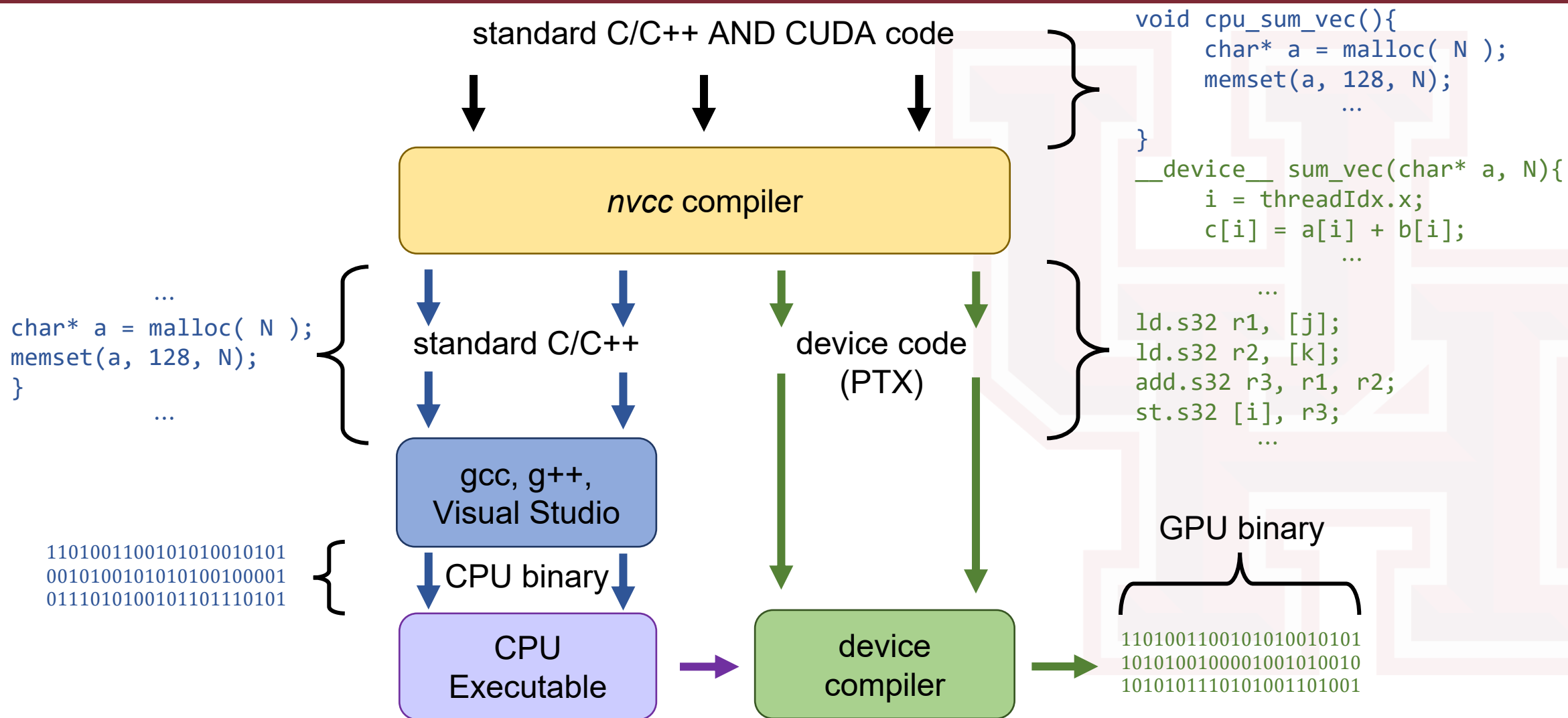
- 5) copy results from the CUDA device to main memory

```
cudaError_t cudaMemcpy(...)
```

CUDA Compiler (nvcc)

- *nvcc* – CUDA compiler, strips CUDA code from C/C++
- CUDA code is converted to device code
 - *parallel thread execution* (PTX) code is compiled just-in-time for available CUDA devices
 - runtime compiler converts PTX into device binary code that can be executed directly on the CUDA devices
- C/C++ code is passed to the native compiler – gcc, g++, Visual Studio, etc.
 - C/C++ is compiled and linked as normal
 - code is linked to CUDA libraries that handle links to device functions
- This operation is usually transparent to the programmer
- PTX can be useful for profiling and optimization

CUDA Compiler (nvcc)



CUDA Thread Resources

- Threads are executed in parallel on the GPU
- Tesla P100 has 56 SMs, each with 64 cores
= 3584 total cores
 - at least this many threads can be executed *simultaneously*
 - additional threads can be accommodated with hyperthreading
- Some resources are shared:
 - *global memory* is shared across all threads
 - *shared memory* is shared across threads running on a single SM
 - *registers* are only available to a single thread
 - proximity $\approx$ speed
 - *global memory* (200 – 400 clock cycle latency)
 - *shared memory* (20 – 40 clock cycle latency)
 - *registers* are fastest ($\approx$ 11 clock cycles for a calculation involving registers)

Declaring Kernel Functions

- CUDA contains three additional keywords for function declarations
- Declarations specify how the code is to be compiled by nvcc

`__host__`
`__device__`
`__global__`

called and executed on the host
called and executed on a device
called on the host, executed on a device

- These keywords start the function declaration:

```
__host__ int abs(int v){  
    if(v < 0) return -v  
    return v;  
}
```

```
__device__ int abs(int v){  
    if(v < 0) return -v  
    return v;  
}
```

__host__ Functions

- Mostly ignored by nvcc
 - kernels can be launched from host functions
 - kernel launches are stripped from the C/C++ and replaced with links to CUDA launch instructions
- host functions can only be called by other host functions
- functions without CUDA declarations are assumed to be __host__ functions
- A standard C function:

```
int abs(int v){  
    if(v < 0) return -v;  
    return v;  
}
```

↓ is identical to

```
__host__ int abs(int v){  
    if(v < 0) return -v;  
    return v;  
}
```

__device__ Functions

- Compiled completely by nvcc – not passed to the host compiler
- Only executable on a device
- Only __device__ functions can be called by __device__ functions
- Once you're executing on the device, the only way off is to return
- Functions can be declared as both __host__ and __device__

```
__host__ __device__  
int abs(int v){  
    if(v < 0) return -v;  
    return v;  
}
```

- Two versions of the function are created: one in PTX (by nvcc) and the other by the native compiler

__global__ Functions

- Used to specify kernels (functions that execute on the GPU)
- Kernels are called by host functions
 - Special parameters are specified to define how the kernel is *launched*
 - Once launched, the kernel runs on the device
 - Execution continues in the host function that launched the kernel
- `__global__` functions are the only device functions that can be called from a host function
- `__global__` functions must have a void return type

Demo: cuda-samples

[git@github.com:NVIDIA/cuda-samples.git](https://github.com/NVIDIA/cuda-samples.git)

